

Deviation Management – remaining requirements

The creation screen is complete. This document covers everything after it, in delivery order. Part A is written out in full because every file it touches has already been read. Parts B to G need files first – each one says exactly which.

At a glance

	What	Action
READY	Part A – Deviation List tab, Deviation History tab, delete / acknowledge, popup on a green row (REQ 4 to 9, REQ 11)	Apply Steps 1 to 9 below. No files needed.
NEED	Part B – Manage Filtering Laws dialog	Send 3 files
NEED	Part C – Configuration update warnings (REQ 10)	Send 3 files
NEED	Part D – Deviation applied during flight analysis (REQ 12)	Send 1 search result and 2 files
NEED	Part E – Relearn on a learned threshold	Send 2 files
NEED	Part F – DACOM extraction (REQ 13)	Send 2 files
NEED	Part G – Purge	Send 1 file

The last page lists every file and search in one checklist, with exactly how to find each one in Eclipse.

Part A – Deviation List, Deviation History, delete

One new class is used twice, once per tab. The List tab has check boxes and a Delete Deviation button; the History tab is read-only. Deleting sets the end date – because the configuration itself was never modified, the indicator reverts to its configuration values automatically, which is REQ 8.

Step 1 — IndicatorDeviationDao – instantiate the law parameters before returning

mgs-services-impl/.../service/health/common/ejb/IndicatorDeviationDao.java

The tabs show the filtering law parameters, and getContextualParameters() is a lazy collection.

HealthCommonBusiness runs on the client, so the collection must be loaded here, on the server, while the session is open – the same pattern as the getFilteringLaws().size() line in HealthDao. Add this private method and call it on the result of both findActive and findHistory.

```
private List<IndicatorDeviation> initialise(List<IndicatorDeviation> deviations) {
    for (IndicatorDeviation d : deviations) {
        if (d.getFilteringLaw() != null) {
            d.getFilteringLaw().getContextualParameters().size(); // force load
        }
    }
    return deviations;
}
```

// in findActive and findHistory, change the last line to:
return initialise(q.getResultList());

Step 2 — IndicatorDeviationDto – one more field

mgs-services-interfaces/.../service/health/common/dto/IndicatorDeviationDto.java

The formatted parameter text for the Filtering Law Parameters column.

```
private String filteringLawParameters;

public String getFilteringLawParameters() { return filteringLawParameters; }
public void setFilteringLawParameters(String p) { this.filteringLawParameters = p; }
```

Step 3 — HealthCommonBusiness → toDtos – format the parameters

Inside the existing if (d.getFilteringLaw() != null) block, after the two setters already there.

```

StringBuilder params = new StringBuilder();
for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
    if (params.length() > 0) {
        params.append(" | ");
    }
    params.append(p.getName())
        .append(", min=").append(p.getMinValue())
        .append(", max=").append(p.getMaxValue());
    if (p.getThresholdOfVariabilityValue() != null) {
        params.append(", S=").append(p.getThresholdOfVariabilityValue());
    }
}
dto.setFilteringLawParameters(params.toString());

```

Import `com.eurocopter.mgs.datamodel.health.monitoring.ContextualParameter`. If `getThresholdOfVariabilityValue()` returns a primitive double the null check will not compile – drop the if and append it always. S is only set on HELIONIX, so on M'ARMS it will read 0.

Step 4 — HealthAlarmsPreferenceStore – four new members

`mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java`

A map from indicator id to its display path, and three thin pass-throughs the tabs call.

```

// field, beside lawNameToId
private final Map<String, String> indicatorDisplayNames = new HashMap<>();

public String getIndicatorDisplayName(String indicatorId) {
    String name = indicatorDisplayNames.get(indicatorId);
    return name != null ? name : "";
}

public List<IndicatorDeviationDto> findActiveDeviations(Aircraft aircraft) {
    if (aircraft == null) {
        return new ArrayList<>();
    }
    try {
        return healthBusiness.findActiveDeviations(aircraft.getId());
    } catch (Exception e) {
        LOGGER.warn("Could not load active deviations", e);
        return new ArrayList<>();
    }
}

public List<IndicatorDeviationDto> findDeviationHistory(Aircraft aircraft) {
    if (aircraft == null) {
        return new ArrayList<>();
    }
    try {
        return healthBusiness.findDeviationHistory(aircraft.getId());
    } catch (Exception e) {
        LOGGER.warn("Could not load deviation history", e);
        return new ArrayList<>();
    }
}

public void acknowledgeDeviations(List<String> deviationIds) {
    try {
        healthBusiness.acknowledgeDeviations(deviationIds);
    } catch (Exception e) {
        LOGGER.error("Could not acknowledge deviations", e);
    }
}

```

Step 5 — HealthAlarmsPreferenceStore → getThresholdModels – fill the display map

Inside the `for (Indicator indicator : sortedSet)` loop, as its first line. The requirement asks for the component path, not just the indicator name, because the name alone is not unique.

```

String path = healthMechanicalComponent.getPath()
    .replace(String.valueOf(HealthMechanicalComponent.HMC_PATH_SEPARATOR), " - ");
if (path.startsWith(" - ")) {
    path = path.substring(3);
}
indicatorDisplayNames.put(indicator.getId(), path + " - " + indicator.getName());

```

Step 6 — DeviationTableComposite – new class

mgs-health-app/.../preferences/healthalarms/DeviationTableComposite.java

Same package as HealthAlarmsComposite. history=false gives the List tab with check boxes and Delete; history=true gives the read-only History tab with a date range.

```
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms;

import java.text.MessageFormat;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.List;

import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.jface.viewers.ArrayContentProvider;
import org.eclipse.jface.viewers.CheckboxTableViewer;
import org.eclipse.jface.viewers.ITableColorProvider;
import org.eclipse.jface.viewers.ITableLabelProvider;
import org.eclipse.jface.viewers.LabelProvider;
import org.eclipse.jface.viewers.TableViewer;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Color;
import org.eclipse.swt.graphics.Image;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Button;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Table;
import org.eclipse.swt.widgets.TableColumn;

import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.core.presentation.utils.ConvertUtils;
import com.eurocopter.mgs.core.presentation.utils.CoreResourcesUtils;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store.HealthAlarmsPreferenceStore;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;

public class DeviationTableComposite extends Composite {

    private static final String[] COLUMN_KEYS = { "deviationColIndicator", "Severity",
        "ThresholdLevel", "Type", "LearningThresholdMinValue", "LearningThresholdMaxValue",
        "deviationColPoints", "FilteringLaw", "deviationColLawParameters", "deviationColTe",
        "deviationColDate" };
    private static final int[] COLUMN_WIDTHS = { 320, 70, 60, 70, 80, 80, 60, 100, 280, 150, 170 };

    private final HealthAlarmsPreferenceStore store;
    private final boolean history;
    private final Runnable onChanged;
    private final TableViewer viewer;
    private final SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
    private Aircraft aircraft;

    public DeviationTableComposite(Composite parent, HealthAlarmsPreferenceStore store,
        boolean history, Runnable onChanged) {
        super(parent, SWT.NONE);
        this.store = store;
        this.history = history;
        this.onChanged = onChanged;
        setLayout(new GridLayout(1, false));

        if (!history) {
            Button delete = new Button(this, SWT.PUSH);
            delete.setText(Messages.CORE.getString("deviationDeleteButton"));
            delete.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false));
            delete.addListener(SWT.Selection, e -> deleteSelected());
        }
    }
}
```

```

int style = SWT.BORDER | SWT.FULL_SELECTION | SWT.H_SCROLL | SWT.V_SCROLL
    | (history ? SWT.SINGLE : SWT.MULTI | SWT.CHECK);
Table table = new Table(this, style);
table.setHeaderVisible(true);
table.setLinesVisible(true);
table.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true));
for (int i = 0; i < COLUMN_KEYS.length; i++) {
    TableColumn column = new TableColumn(table, SWT.LEFT);
    column.setText(Messages.CORE.getString(COLUMN_KEYS[i]));
    column.setWidth(COLUMN_WIDTHS[i]);
}

viewer = history ? new TableViewer(table) : new CheckboxTableViewer(table);
viewer.setContentProvider(ArrayContentProvider.getInstance());
viewer.setLabelProvider(new DeviationLabelProvider());
}

public void setAircraft(Aircraft aircraft) {
    this.aircraft = aircraft;
    reload();
}

public void reload() {
    viewer.setInput(history
        ? store.findDeviationHistory(aircraft)
        : store.findActiveDeviations(aircraft));
}

private void deleteSelected() {
    Object[] checked = ((CheckboxTableViewer) viewer).getCheckedElements();
    if (checked.length == 0) {
        return;
    }
    List<String> ids = new ArrayList<>();
    List<String> names = new ArrayList<>();
    for (Object o : checked) {
        IndicatorDeviationDto d = (IndicatorDeviationDto) o;
        ids.add(d.getId());
        names.add(store.getIndicatorDisplayName(d.getIndicatorId()));
    }
    boolean confirmed = MessageDialog.openConfirm(getShell(),
        Messages.CORE.getString("deviationDeleteTitle"),
        MessageFormat.format(Messages.CORE.getString("deviationDeleteConfirm"),
            String.join(", ", names)));
    if (!confirmed) {
        return;
    }
    store.acknowledgeDeviations(ids);
    onChanged.run();
}

private String number(Number value) {
    return value == null ? "" : ConvertUtils.formatNumber(value, -1);
}

private final class DeviationLabelProvider extends LabelProvider
    implements ITableLabelProvider, ITableColorProvider {

    @Override

```

```

    public Image getColumnImage(Object element, int column) {
        return null;
    }

    @Override
    public String getColumnText(Object element, int column) {
        IndicatorDeviationDto d = (IndicatorDeviationDto) element;
        boolean fixed = "FIXED".equals(d.getThresholdType());
        switch (column) {
            case 0: return store.getIndicatorDisplayName(d.getIndicatorId());
            case 1: return d.getSeverity();
            case 2: return d.getThresholdLevel();
            case 3: return d.getThresholdType();
            case 4: return number(d.getMinValue());
            case 5: return number(fixed ? d.getThresholdValue() : d.getMaxValue());
            case 6: return number(d.getNbSuccessivePoints());
            case 7: return d.getFilteringLawName() != null ? d.getFilteringLawName() : "";
            case 8: return d.getFilteringLawParameters() != null ? d.getFilteringLawParameters() : "";
            case 9: return d.getTeReference();
            case 10: return history && d.getEndDate() != null
                ? dateFormat.format(d.getStartDate()) + " - " + dateFormat.format(d.getEndDate())
                : dateFormat.format(d.getStartDate());
            default: return "";
        }
    }

    @Override
    public Color getBackground(Object element, int column) {
        if (column != 1) {
            return null;
        }
        String severity = ((IndicatorDeviationDto) element).getSeverity();
        if ("RED".equals(severity)) {
            return CoreResourcesUtils.getColor(CoreResourcesUtils.COLOR_RED);
        }
        if ("AMBER".equals(severity)) {
            return CoreResourcesUtils.getColor(CoreResourcesUtils.COLOR_AMBER);
        }
        return null;
    }

    @Override
    public Color getForeground(Object element, int column) {
        return null;
    }
}

```

ConvertUtils.formatNumber is the same helper WCTreeLabelProvider already uses. Check its import path against that class – it may live in a different package from the one shown.

Step 7 — HealthAlarmsComposite – wire the two tabs

mgs-health-app/.../preferences/healthalarms/HealthAlarmsComposite.java

Four edits in this class.

a. Two fields, beside thresholdsTreeViewer:

```

private DeviationTableComposite deviationListComposite;
private DeviationTableComposite deviationHistoryComposite;

```

b. In the tab block, replace the two empty composites:

```

// BEFORE
deviationListTab.setControl(new Composite(tabFolder, SWT.NONE));
deviationHistoryTab.setControl(new Composite(tabFolder, SWT.NONE));

// AFTER
deviationListComposite = new DeviationTableComposite(tabFolder,
    healthAlarmsPreferenceStore, false, this::reloadAll);
deviationListTab.setControl(deviationListComposite);

deviationHistoryComposite = new DeviationTableComposite(tabFolder,
    healthAlarmsPreferenceStore, true, this::reloadAll);
deviationHistoryTab.setControl(deviationHistoryComposite);

```

c. In the aircraft selection listener, after refreshViewer(selectedAircraft, aircraftComponentModel);:

```
deviationListComposite.setAircraft(selectedAircraft);
deviationHistoryComposite.setAircraft(selectedAircraft);
```

The tree must load first: it fills the indicator display map the tabs read.

d. New method, and call it after a confirmed Apply. Reloading after the save also replaces the "SAVED" placeholder with the real id from the database.

```
private void reloadAll() {
    if (selectedAircraft == null) {
        return;
    }
    healthWcModelMap.clear();
    refreshViewer(selectedAircraft, healthAlarmsPreferenceStore.loadHealthModels(selectedAircraft));
    deviationListComposite.setAircraft(selectedAircraft);
    deviationHistoryComposite.setAircraft(selectedAircraft);
}

// in performApply, confirmed branch:
if (MgsPreferenceStoreManager.wasLastSaveConfirmed()) {
    healthAlarmsPreferenceStore.saveDeviations();
    MgsPreferenceStoreManager.clearAircraftConfigurations();
    reloadAll(); // <<< add
}
```

Step 8 — HealthAlarmsComposite → createTreeViewer – popup on a deviated row

The client asked for a message when the user clicks a green row, so they understand why it cannot be edited. Add at the end of createTreeViewer.

```
tree.addListener(SWT.MouseDown, event -> {
    TreeItem item = tree.getItem(new Point(event.x, event.y));
    if (item != null && item.getData() instanceof WCThresholdModel model && model.isDeviated()) {
        MessageDialog.openInformation(tree.getShell(),
            Messages.CORE.getString("deviationAlreadySetTitle"),
            Messages.CORE.getString("deviationAlreadySet"));
    }
});
```

Imports: org.eclipse.swt.widgets.TreeItem, org.eclipse.swt.graphics.Point, org.eclipse.jface.dialogs.MessageDialog, and WCThresholdModel.

Step 9 — CoreMessages.properties – and every language file

New keys only. The other column headers reuse the Health tree already has.

```
deviationColIndicator=Indicator
deviationColPoints=Points
deviationColLawParameters=Filtering Law Parameters
deviationColTe=TE
deviationColDate=Date
deviationDeleteButton=Delete Deviation
deviationDeleteTitle=Delete deviation
deviationDeleteConfirm=Are you sure you want to delete the deviation set on indicator {0}
deviationAlreadySetTitle=Deviation
```

Build: mgs-services-interfaces, mgs-services-impl, mgs-health-business, client. Redeploy once.

Test: save two deviations, open Deviation list – both shown with full path. Tick one, Delete, OK – it leaves the list, appears in Deviation history with a date range, and its row in the tree goes back to its configuration values and is editable again.

Decision recorded: the list shows all thirteen parameters for now. REQ 6 asks for only the modified ones; that is deferred and will be added later if the client asks for it. Log it in the tracker so it is not lost.

Part B – Manage Filtering Laws dialog

A TitleAreaDialog opened from a button beside Apply. Top: all laws with their contextual parameters. Bottom: a form with a name and CTXT1 to CTXT5, each a parameter name, min, max and stability. Create, Update and Delete. Laws from the configuration file, or referenced by any deviation, are greyed and not selectable. No database change – filtering_law and contextual_parameter already exist.

File needed	Why
FilteringLaw.java (server, mgs-jpa-model)	Its fields and mapping, to write create / update / delete and to know whether it records which laws came from the configuration file
ContextualParameter.java (server)	Exact field names and types, including getThresholdOfVariabilityValue
Any existing TitleAreaDialog in the client	So the new dialog matches the house style for layout, buttons and messages

Also ask: where does the list of allowed contextual parameter names come from? The M'ARMS filtering algorithm hard-codes nine (NR, TRQ1_TRQ2, IAS, ZB and so on). The dropdown should offer the same set.

Part C – Configuration update warnings (REQ 10)

Applying a new configuration file to an aircraft acknowledges every active deviation on it. Warnings: a label on the Update Aircraft window and on the Import configurations dialog, a label on the Threshold learning dialog, and a blocking confirm on the Update Aircraft OK button. The acknowledge itself is one call to acknowledgeDeviations with every active id.

File needed	How to find it
Update Aircraft window	Search *.properties for "Update Aircraft", then the key in *.java
Import configurations dialog	Same approach with "Import configurations"
Threshold learning dialog	Search *.properties for "last downloaded flight hours"

Part D – Applicability by session data date (REQ 12)

What it means. Today a deviation is saved and shown on screen, but when a flight is downloaded MGS still checks the indicator values against the *configuration* thresholds. Part D makes the analysis use the *deviated* values instead, but only for sessions inside the deviation's date range.

Example	Result
Deviation created 1 Feb, session dated 31 Jan	Configuration values
Deviation created 1 Feb, session dated 4 Feb	Deviated values
Deviation deleted 15 Feb, session dated 14 Feb	Deviated values
Deviation deleted 15 Feb, session dated 17 Feb	Configuration values

The date that counts is the session's own data date, not the day it was downloaded. This is the highest-risk part and the one the November date depends on, so the search below should be run first.

Needed	Why
Search result, not files: grep *.java for getValue(), getThresholdLevel(), getMinimumThreshold() across mgs-server	Tells us how many places read a threshold during analysis. One place means a few days; forty means the estimate doubles
The class that compares an indicator value with its threshold during download	Likely beside LearnedThresholdAnalyzer or the filtering algorithms already seen
The Flight or session entity	Where the data date and the offload date are held, so the rule uses the right one

Part E – Relearn on a learned threshold

A deviation on a learned threshold triggers a relearn over the following flight hours, and a relearn request must not delete the deviation. Note that HealthThresholdDelegate already creates a relearn request on every save of a

learned threshold, so part of this may already happen.

File needed	Why
LearnedThresholdAnalyzer	How learning reads its bounds, so it can read deviated ones
The class that processes RelearnThresholdRequest during download	To confirm it never touches indicator_deviation

Part F – DACOM extraction (REQ 13)

New GSC_DEVIATIONS.sql for active deviations only, and the existing THRESHOLDS extraction carrying deviated values with no format change. With the separate table the first part is one SELECT.

File needed	Why
One existing GSC_*.sql, ideally the thresholds one	Format and conventions
The class that generates the THRESHOLDS file, and whatever schedules it	To add deviated values without changing its output format

Part G – Purge

Delete a deviation once its end date is on or before the start date of the oldest session that has not been purged. Active deviations are never purged.

File needed	Why
The existing session purge job	So deviation purge runs at the same point with the same date

What I need from you – checklist

Send these in this order. Part D's search comes first because it decides the size of the biggest remaining piece.

#	For	What to send	How to find it in Eclipse
1	D	Search result – file names and hit counts, no files	Ctrl+H, File Search, *.java, scope mgs-server. Search getThresholdLevel() then getMaximumThreshold(). Screenshot the Search view.
2	D	The class that compares an indicator value with its threshold during download	Look in the Search view results from item 1 for a class under download or analysis
3	D	The Flight entity	Ctrl+Shift+T, type Flight, pick the one in mgs-jpa-model/src/main/java
4	B	FilteringLaw.java	Ctrl+Shift+T, FilteringLaw, the mgs-jpa-model source
5	B	ContextualParameter.java	Ctrl+Shift+T, ContextualParameter, same
6	B	Any existing TitleAreaDialog	Ctrl+Shift+T, TitleAreaDialog, then Ctrl+T for implementors in mgs-client
7	C	Update Aircraft window	Ctrl+H, *.properties, "Update Aircraft"; take the key, search it in *.java
8	C	Import configurations dialog	Same, with "Import configurations"
9	C	Threshold learning dialog	Same, with "last downloaded flight hours"
10	E	LearnedThresholdAnalyzer	Ctrl+Shift+T, LearnedThresholdAnalyzer
11	E	The class that processes relearn requests	Ctrl+Shift+G on RelearnThresholdRequest, look under download
12	F	One GSC_*.sql file	Ctrl+Shift+R, GSC_ – the thresholds one if there is one
13	F	The THRESHOLDS file generator	Ctrl+H, *.java, THRESHOLDS
14	G	The session purge job	Ctrl+H, *.java, purge

Right now: apply Part A, and send item 1. Item 1 costs two minutes and tells us whether the rest fits before the end of November.